PHANTOM

Autonomous Adversary Simulation Platform

System Design & Technical Report

Version 1.0

Author: Gadouri Rodwan

Supervisor: Dr. Allama Oussama

Institution: ESTIN

Repository: https://github.com/Usta0x001/Phantom

March 2, 2026

Abstract

Phantom is an autonomous AI-powered penetration testing platform that uses large language models (LLMs) to discover, exploit, and verify real vulnerabilities in web applications, APIs, and network services. Unlike traditional vulnerability scanners that rely on static signatures and predefined rules, Phantom employs a ReAct (Reason + Act) reasoning loop that enables adaptive, multi-step attack strategies similar to those used by human security professionals.

The system implements a layered architecture with seven defense-in-depth security controls: scope validation with DNS pinning, a tool invocation firewall with injection detection, sandboxed execution in ephemeral Docker containers, cost and time controllers, HMAC-chained audit logging, and output sanitization. All offensive operations execute within isolated containers with no host filesystem access.

Phantom integrates 30+ security tools (nmap, nuclei, sqlmap, ffuf, Playwright, and others) through a unified execution framework and supports 100+ LLM providers via LiteLLM. The system has been validated through 808+ automated tests and two deep security audits that identified and resolved 83 findings across all severity levels.

This report presents the complete system design, architecture, security model, tool ecosystem, and testing methodology of the Phantom platform.

Keywords: penetration testing, autonomous agents, large language models, vulnerability discovery, security automation, adversary simulation

Contents

Abstract **1**

1 Introduction **2**
1.1 Context and Motivation 2
1.2 Objectives 2
1.3 Contributions 3
1.4 Report Organization 3

2 System Architecture **4**
2.1 Architecture Overview 4
2.2 Package Structure 4
2.3 Design Principles 5

3 Core Components **6**
3.1 BaseAgent . . . 6
3.2 AgentState . . . 6
3.3 EnhancedAgentState . . . 7
3.4 LLM Client . . . 7
3.5 Memory Compressor . . . 7

4 Security Architecture **8**
4.1 Layer 1: Scope Validator . . . 8
4.2 Layer 2: Tool Invocation Firewall . . . 8
4.3 Layer 3: Docker Sandbox . . . 8
4.4 Layer 4: Cost Controller . . . 9
4.5 Layer 5: Time Controller . . . 9
4.6 Layer 6: Audit Logger . . . 9
4.7 Layer 7: Output Sanitizer . . . 9

5 LLM Integration **10**
5.1 Provider Support . . . 10
5.2 Prompt Architecture . . . 10
5.3 Streaming and Cost Tracking . . . 10

6 Tool Ecosystem **11**
6.1 Tool Categories . . . 11
6.2 Tool Execution Path . . . 11
6.3 Tool Registry . . . 12

7 Data Models **13**
7.1 Vulnerability Model . . . 13
7.2 Host Model . . . 13
7.3 Scan Model . . . 13

8 Scan Lifecycle **14**
8.1 Initialization . . . 14
8.2 Execution (ReAct Loop) . . . 14
8.3 Finalization . . . 15

9 Report Pipeline **16**
9.1 Report Formats . . . 16
9.2 Output Structure . . . 16
9.3 Credential Scrubbing . . . 17

10 Skills and Knowledge System **18**
10.1 Skills . . . 18
10.2 Knowledge Persistence . . . 18

11 Testing and Quality Assurance **19**
11.1 Test Suite . . . 19
11.2 Security Audit . . . 19

12 Performance and Scalability **20**

12.1 Resource Bounds 20
12.2 Optimization Techniques 20

13 Future Work **21**
13.1 Async-First Architecture 21
13.2 Plugin-Based Tool System 21
13.3 State Machine Formalization 21
13.4 Multi-Agent Orchestration 21
13.5 Benchmark Suite 21

14 Conclusion **22**

Chapter 1

Introduction

1.1 Context and Motivation

Modern web applications are increasingly complex, employing dynamic frontends, microservice backends, containerized deployments, and cloud-native architectures. This complexity creates attack surfaces that are difficult to assess through traditional vulnerability scanning approaches.

Traditional scanners operate through signature matching and predefined rule sets. While effective for known vulnerability patterns, they fundamentally struggle with:

- **Multi-step vulnerabilities** that require chaining multiple attack vectors across different application components
- **Business logic flaws** that no signature database can capture
- **Novel attack surfaces** in REST APIs, GraphQL endpoints, WebSockets, and single-page applications
- **Context-dependent vulnerabilities** that require understanding application behavior and state
- **Authentication-gated functionality** that requires session management and credential handling

Manual penetration testing addresses these limitations but is expensive, time-consuming, and does not scale. A single comprehensive pentest can cost $10,000–$50,000 and take weeks to complete. Organizations that need continuous security assessment cannot afford this cadence.

1.2 Objectives

Phantom was designed to address the gap between automated scanners and manual pentesting by introducing autonomous AI reasoning into the security testing workflow. The primary design objectives are:

1. **Autonomous operation:** Execute complete penetration tests with zero human intervention

2. **Real vulnerability discovery:** Find exploitable bugs, not theoretical risks
3. **Verification:** Confirm every finding by independently re-exploiting it
4. **Safety:** Strict scope enforcement, sandboxed execution, and complete audit trail
5. **Extensibility:** Skills system for domain-specific knowledge injection
6. **Cost control:** Hard budget limits to prevent runaway LLM spending

1.3 Contributions

The main contributions of this work are:

- A ReAct-based autonomous agent architecture for penetration testing
- A 7-layer defense-in-depth security model for safe autonomous offensive operations
- A unified tool execution framework supporting 30+ security tools in sandboxed environments
- A knowledge persistence system enabling cross-scan learning
- Comprehensive validation through 808+ automated tests and two security audits

1.4 Report Organization

This report is organized as follows: Chapter 2 presents the system architecture. Chapter 3 details the core components. Chapter 4 describes the security architecture. Chapter 5 covers the LLM integration strategy. Chapter 6 documents the tool ecosystem. Chapter 7 explains the data models. Chapter 8 walks through the scan lifecycle. Chapter 9 describes the report pipeline. Chapter 10 covers the skills and knowledge system. Chapter 11 presents the testing methodology and quality metrics. Chapter 12 discusses performance characteristics. Chapter 13 outlines future work. Chapter 14 concludes the report.

Chapter 2

System Architecture

2.1 Architecture Overview

Phantom follows a layered architecture with clear separation of concerns. The system is organized into six major layers, each with distinct responsibilities:

Table 2.1: Architecture Layers

Layer	Components	Responsibility
Interface	CLI, TUI, Streaming Parser	User interaction and live display
Orchestration	Profile, Scope, Cost, Audit	Scan configuration and governance
Agent Core	BaseAgent, State, LLM, Memory	Reasoning loop and state management
Security	Firewall, Verifier, Scope	Input validation and access control
Execution	Docker, Tool Server, Tools	Sandboxed tool execution
Output	Reports, Graphs, Templates	Structured result generation

2.2 Package Structure

The codebase is organized into 11 Python packages under the `phantom/` namespace, totaling 80+ modules and 15,000+ lines of production code.

Table 2.2: Package Organization

Package	Modules	Purpose
`agents/`	5	Agent core, state machine, ReAct loop
`core/`	20	Security, verification, reporting, compliance
`tools/`	25+	Tool implementations and dispatch
`llm/`	7	LLM client, memory compression, providers
`runtime/`	4	Docker sandbox and tool server
`models/`	5	Pydantic domain models
`skills/`	50+	Domain knowledge Markdown files
`interface/`	8	CLI, TUI, formatters
`telemetry/`	2	Run tracing and statistics
`config/`	1	Environment-based configuration
`utils/`	1	Resource path resolution

2.3 Design Principles

The architecture follows several key design principles:

Defense in Depth: Every tool invocation passes through multiple security gates before reaching the execution layer. No single point of failure can compromise system safety.

Bounded Resources: All collections (messages, findings, actions, errors) have hard upper limits with FIFO eviction to prevent unbounded memory growth.

Crash Safety: Partial results are preserved if the LLM fails mid-scan. Checkpoint-based resume enables continuation after failures.

Separation of Concerns: Local agent logic (reasoning, state management) is cleanly separated from remote execution (sandbox tools, Docker).

Auditability: Every tool invocation, finding, and agent decision is logged to an HMAC-chained audit trail for tamper-evident record keeping.

Chapter 3

Core Components

3.1 BaseAgent

The `BaseAgent` class (859 lines, `agents/base_agent.py`) implements the ReAct reasoning loop that drives all agent behavior. On each iteration, the agent:

1. Sends the conversation history (including the findings ledger) to the LLM
2. Parses tool invocations from the LLM response (XML format)
3. Validates each invocation through the Tool Firewall
4. Executes approved tools (local or sandbox)
5. Records findings and updates agent state
6. Checks stop conditions (iterations, time, cost, agent finish)

The agent supports graceful crash handling: if the LLM fails mid-scan, partial results are saved and a report is generated from whatever findings have been collected.

3.2 AgentState

`AgentState` (`agents/state.py`) is a Pydantic model that tracks the agent's complete state with bounded collections:

Table 3.1: AgentState Resource Bounds

Collection	Limit	Behavior at Limit
Messages	500	FIFO eviction, system prompt preserved
Findings Ledger	200	Hard cap, NEVER compressed
Actions	5,000	FIFO eviction
Errors	1,000	FIFO eviction
Iterations	200	Scan terminates
Wall-clock time	4 hours	Cumulative across resumes

3.3 EnhancedAgentState

Extends `AgentState` with vulnerability tracking, host discovery, scan phase management, endpoint deduplication, and priority queuing. Provides `checkpoint save/restore` for resumable scans and `to_report_data()` export for structured reporting.

3.4 LLM Client

The LLM client (`llm/llm.py`) wraps LiteLLM for multi-provider access supporting OpenRouter, OpenAI, Anthropic, Google, Groq, DeepSeek, Ollama, and 100+ other providers. Key features include:

- Async streaming for real-time CLI display
- Per-request cost tracking (input, output, and cached tokens)
- Prompt caching support for compatible providers
- Vision/screenshot support for browser-based testing
- Automatic retries with exponential backoff
- Credential redaction before sending prompts

3.5 Memory Compressor

When conversation exceeds 80,000 tokens, the `MemoryCompressor` (`llm/memory_compressor.py`):

1. Keeps the system prompt and last 12 messages verbatim
2. Summarizes older messages via a separate LLM call
3. Preserves all URLs, payloads, credentials, and vulnerability details in summaries
4. The `findings_ledger` is NEVER compressed — it serves as permanent agent memory

Chapter 4

Security Architecture

Phantom implements 7 layers of defense-in-depth to prevent misuse, scope violations, and unintended damage. Every tool invocation passes through multiple security gates before reaching the execution layer.

4.1 Layer 1: Scope Validator

The `ScopeValidator` enforces whitelist-based target authorization with DNS pinning. It supports domain, IP, CIDR, and regex scope rules. Private IP detection prevents SSRF attacks against internal networks. DNS resolution is cached (bounded to 10,000 entries) and pinned to prevent DNS rebinding attacks. Validation occurs on every tool call, not just at scan start.

4.2 Layer 2: Tool Invocation Firewall

The `ToolInvocationFirewall` validates every tool call before execution:

- Checks 8 injection patterns: semicolons, pipes, backticks, `$()`, `${}`, redirections, `&&`, `||`
- Enforces per-tool argument whitelists for `nmap`, `sqlmap`, `ffuf` extra arguments
- Rejects arguments longer than 4,096 characters
- Blocks dangerous sandbox commands: fork bombs, `rm -rf /`, `curl|sh`
- Logs all violations to the HMAC-chained audit trail

4.3 Layer 3: Docker Sandbox

All offensive tools execute inside an ephemeral Docker container with:

- No host filesystem access
- Restricted Linux capabilities

- Port reservation with minimal TOCTOU window (socket held until Docker binds)
- Symlink-safe file operations
- Tar archive size cap (500 MB) to prevent disk bombs
- Automatic cleanup on scan completion or failure

4.4 Layer 4: Cost Controller

The `CostController` enforces hard budget limits (default $25) with an 80% warning threshold. Every LLM request tracks input tokens, output tokens, cached tokens, and cost. Per-request ceiling prevents individual expensive calls.

4.5 Layer 5: Time Controller

Wall-clock time limit of 4 hours, cumulative across scan resumes via `_cumulative_elapsed_seconds`. Prevents runaway scans that could consume resources indefinitely.

4.6 Layer 6: Audit Logger

The `AuditLogger` provides a crash-safe, tamper-evident audit trail in JSONL format. Each entry is HMAC-SHA256 chained to the previous entry. On scan resume, chain integrity is verified to detect tampering. File rotation at configurable size limits.

4.7 Layer 7: Output Sanitizer

Before report generation, all output passes through credential scrubbing (regex detection of `password=`, `token=`, `api_key=`, `bearer`, `session_id`). CSV formula injection is prevented by prefixing dangerous cell values. Recursive dictionary scrubbing handles nested structures.

Table 4.1: Security Layer Summary

Layer	Component	Protection
L1	Scope Validator	Target authorization, DNS pinning, SSRF prevention
L2	Tool Firewall	Injection detection, argument whitelisting
L3	Docker Sandbox	Process isolation, no host access
L4	Cost Controller	Budget limits ($25 default)
L5	Time Controller	Wall-clock limit (4h cumulative)
L6	Audit Logger	HMAC chain, tamper detection
L7	Output Sanitizer	Credential scrubbing, CSV injection prevention

Chapter 5

LLM Integration

5.1 Provider Support

Phantom uses LiteLLM to support 100+ LLM providers through a unified API. The recommended configuration uses OpenRouter as a gateway to access DeepSeek, Claude, GPT-4, and Llama models. Direct provider support includes OpenAI, Anthropic, Google, Groq, and local inference via Ollama or vLLM.

5.2 Prompt Architecture

The agent's system prompt is assembled from five components:

1. **Persona definition:** PhantomAgent identity, rules, and behavioral constraints
2. **Tool schemas:** Available tool definitions in XML format
3. **Loaded skills:** Domain-specific knowledge (Markdown files selected based on target)
4. **Scan profile directives:** Depth, iteration limits, tool restrictions
5. **Findings ledger:** Permanent record of all discoveries (never compressed)

Jinja2 templates with autoescape are used for safe prompt construction, preventing injection through template variables.

5.3 Streaming and Cost Tracking

LLM responses are streamed via async iterators for real-time CLI display. Every request tracks: input tokens, output tokens, cached tokens, and cost in USD. The cost controller enforces both per-scan and per-request budget limits. Credential patterns are redacted from prompts before transmission to the LLM provider.

Chapter 6

Tool Ecosystem

6.1 Tool Categories

Phantom provides 30+ security tools organized into functional categories:

Table 6.1: Tool Categories

Category	Tools	Execution Mode
Reconnaissance	nmap, httpx, subfinder	Docker Sandbox
Discovery	ffuf, nuclei, gobuster	Docker Sandbox
Exploitation	sqlmap, nikto, custom scripts	Docker Sandbox
Browser	Playwright (navigate, click, JS)	Docker Sandbox
Agent	sub_agent, todo, notes, finish	Local Process
Reporting	record_finding, web_search	Local Process

6.2 Tool Execution Path

The tool execution pipeline follows a strict sequence:

1. LLM response is parsed for XML tool invocations
2. Each invocation is validated by the `ToolFirewall` (scope, injection patterns, argument whitelist)
3. Blocked calls return a descriptive error to the LLM for self-correction
4. Local tools execute in-process; sandbox tools are dispatched via HTTP to the Docker container
5. Results are truncated to 8 KB, XML-wrapped, and appended to the conversation
6. The audit logger records each tool call with timing and result summary

6.3 Tool Registry

Tools are registered at import time using `@tool` decorators with XML schema files. Each schema defines: tool name, description, parameters (type, required, description), and execution mode (local vs. sandbox). The registry supports dynamic content injection for skills-based tool discovery.

Chapter 7

Data Models

Phantom uses Pydantic models for type-safe data management throughout the system.

7.1 Vulnerability Model

Identity: id, name, vulnerability_class, severity (Critical/High/Medium/Low/Info), CVSS score (0–10)

Evidence: Raw HTTP request/response data, payloads used, screenshots

Verification: Status (detected/verified/false_positive), verification method, PoC code

Enrichment: CWE IDs, MITRE ATT&CK techniques, OWASP category, remediation steps

7.2 Host Model

Identity: IP address, hostname, OS fingerprint

Services: Port number, protocol, service name, version, state

Technologies: Name, version, category (framework, CMS, server, language)

7.3 Scan Model

ScanResult: scan_id, target, status, timing, vulnerability count

ScanPhase: RECON, ENUMERATION, EXPLOITATION, POST_EXPLOITATION, REPORTING

ScanStatus: INITIALIZING, RUNNING, COMPLETED, FAILED, CANCELLED

Chapter 8

Scan Lifecycle

8.1 Initialization

The scan initialization phase performs the following steps:

1. CLI parses target URLs and scan profile selection
2. `ScopeValidator` created with target whitelist
3. `DockerRuntime` creates ephemeral sandbox container
4. `AuditLogger` and `Tracer` initialized (HMAC chain started)
5. `ToolFirewall` initialized with scope validator reference
6. `CostController` initialized with budget limit
7. `KnowledgeStore` loaded from previous scans

8.2 Execution (ReAct Loop)

Each iteration of the reasoning loop:

1. LLM receives conversation history including the findings ledger
2. Agent reasons about current state and selects a strategy
3. Tool calls are emitted in XML format
4. Executor validates through the firewall and dispatches approved calls
5. Results are recorded and state is updated
6. Memory compression triggers when context exceeds 80,000 tokens
7. Loop detector prevents repetitive behavior patterns

Stop conditions: iteration limit (200), time limit (4 hours), cost limit ($25), agent calling `finish`, or user interrupt.

8.3 Finalization

The post-scan pipeline performs 8 stages:

1. Verification engine re-tests HIGH and CRITICAL findings
2. MITRE ATT&CK enrichment tags each vulnerability with TTPs
3. Compliance mapper generates OWASP, CWE, and NIST reports
4. Attack graph built with NetworkX (nodes: hosts, services, vulnerabilities)
5. Nuclei templates generated for each finding (YAML format)
6. Knowledge store updated with new findings (optional Fernet encryption)
7. Reports generated: JSON + HTML + Markdown (all credential-scrubbed)
8. Sandbox container destroyed, audit trail finalized

Chapter 9

Report Pipeline

9.1 Report Formats

Table 9.1: Report Output Formats

Format	Purpose	Features
JSON	Machine-readable	Full vulnerability data, CVSS, evidence, MITRE mapping
HTML	Executive/team	Styled CSS, sortable tables, severity badges
Markdown	Documentation	GitHub-compatible, CSV-injection safe

9.2 Output Structure

Each scan generates a run directory under `phantom_runs/` containing:

- `scan_stats.json` — Timing, token usage, cost breakdown
- `vulnerabilities.csv` — Quick reference vulnerability index
- `report.json/html/md` — Full structured reports
- `attack_graph.json` — NetworkX graph data
- `compliance_report.json` — OWASP, CWE, NIST mappings
- `nuclei_templates/` — Generated YAML templates
- `audit.jsonl` — HMAC-chained audit trail
- `screenshots/` — Browser capture artifacts

9.3 Credential Scrubbing

Before report generation, all output passes through recursive credential scrubbing. Regular expressions detect patterns including `password=`, `token=`, `api_key=`, `bearer`, and `session_id`, replacing values with `[REDACTED]`. CSV formula injection is prevented by prefixing cell values starting with `=`, `+`, `-`, or `@` with a single quote character.

Chapter 10

Skills and Knowledge System

10.1 Skills

Skills are Markdown files loaded into the agent's system prompt to provide domain expertise. Categories include: reconnaissance, vulnerabilities, frameworks, technologies, protocols, cloud, scan modes, coordination, and custom. The agent selects relevant skills based on the target technology stack and scan profile.

Custom skills can be added to `phantom_knowledge/custom/` to extend the agent's expertise for specific domains or target environments.

10.2 Knowledge Persistence

The `KnowledgeStore` saves discovered hosts, vulnerabilities, false positive signatures, and scan history across sessions. This enables the agent to:

- Skip known false positives from previous scans
- Prioritize previously vulnerable endpoints
- Build a cumulative picture of the target's attack surface
- Track vulnerability lifecycle (new, recurring, resolved)

Optional Fernet encryption protects stored knowledge data at rest.

Chapter 11

Testing and Quality Assurance

11.1 Test Suite

Phantom maintains a comprehensive test suite with 808+ tests, 0 failures, and 21 skipped:

Table 11.1: Test Suite Breakdown

Suite	Tests	Scope
`test_e2e_system.py`	184	Full system integration
`test_v0920_audit_fixes.py`	39	Security fix verification
`test_all_modules.py`	∼200	Module-level unit tests
`test_v0918_features.py`	∼100	Feature regression
`test_v0910_coverage.py`	∼80	Coverage gap tests
`test_security_fixes.py`	∼50	Security-specific tests

11.2 Security Audit

The system underwent two deep offensive security audits:

- **v0.9.19 audit:** 83 findings (8 Critical, 19 High, 34 Medium, 22 Low)
- **Remediation:** All Critical and High findings fixed and verified with dedicated tests
- **System score:** Improved from 5.8/10 to 8.0/10 after full remediation

Critical findings addressed include: command injection via tool arguments, unbounded state growth, race conditions in port reservation, HMAC chain bypass, and ReDoS in URL parsing.

Chapter 12

Performance and Scalability

12.1 Resource Bounds

Table 12.1: System Resource Bounds

Resource	Limit	Purpose
Messages	500	Prevent OOM from conversation growth
Findings	200	Bound permanent ledger
Actions	5,000	Bound tool history
Errors	1,000	Bound error log
Iterations	200	Prevent infinite loops
Wall-clock	4 hours	Prevent runaway scans
Cost	$25	Prevent budget overrun
Argument length	4,096 chars	Prevent injection via overflow
Tar archive	500 MB	Prevent disk bombs
DNS cache	10,000 entries	Prevent cache memory leak
Screenshot	10 MB	Prevent memory bomb via images

12.2 Optimization Techniques

- **Lazy debug logging:** `%`-formatting avoids eager f-string evaluation in hot paths
- **Module-level constants:** Tool maps and regex patterns compiled once at import time
- **Lightweight state summaries:** Hot-path code uses `dict`, not `model_dump()`
- **Bounded collections:** FIFO eviction prevents unbounded memory growth
- **Truncated tool results:** 8 KB cap prevents token waste on verbose tool output

Chapter 13

Future Work

The current architecture (v1) has been hardened through two security audits and achieves a system score of 8.0/10. Architecture v2 will focus on the following areas:

13.1 Async-First Architecture

Replace the synchronous enrichment pipeline with fully asynchronous execution. Enable parallel tool execution with configurable concurrency limits. Non-blocking report generation to reduce scan completion time.

13.2 Plugin-Based Tool System

Replace the hardcoded tool registry with plugin discovery via Python entry points. Enable third-party tool packages that can be installed independently. Support hot-reload of tool schemas without system restart.

13.3 State Machine Formalization

Replace ad-hoc phase tracking with a formal state machine implementation. Enable checkpoint schema versioning for forward compatibility across system updates.

13.4 Multi-Agent Orchestration

Implement hierarchical agent delegation with shared context passing. Develop specialized agents: `ReconAgent`, `ExploitAgent`, `ReportAgent`. Enable inter-agent message passing with priority-based routing.

13.5 Benchmark Suite

Create a standardized benchmark suite using intentionally vulnerable applications (DVWA, Juice Shop, WebGoat) to measure detection rate, false positive rate, and time-to-finding across system versions.

Chapter 14

Conclusion

Phantom represents a new approach to automated security testing by combining large language model reasoning with a hardened execution sandbox and comprehensive security architecture. The system demonstrates that autonomous AI agents can discover real vulnerabilities that traditional signature-based scanners miss, while maintaining safety through strict scope enforcement, sandboxed execution, and tamper-evident audit logging.

The system has been validated through 808+ automated tests, two deep security audits, and iterative development across 20 release versions. All 83 security findings identified during auditing have been resolved and verified with dedicated regression tests, achieving a system score of 8.0/10.

The foundation built in version 1 is production-grade and ready for evolution toward Architecture v2, which will bring async-first execution, plugin-based tool management, and hierarchical multi-agent orchestration. The path forward is to benchmark the system against real-world intentionally vulnerable applications to measure and improve detection capabilities before architectural evolution.

Phantom — Autonomous Adversary Simulation Platform
Author: Gadouri Rodwan
Supervisor: Dr. Allama Oussama

Bibliography

[1] Yao, S., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR*, 2023.

[2] BerriAI. "LiteLLM: Call All LLM APIs Using the OpenAI Format." https://github.com/BerriAI/litellm, 2024.

[3] ProjectDiscovery. "Nuclei: Fast and Customizable Vulnerability Scanner." https://github.com/projectdiscovery/nuclei, 2024.

[4] OWASP Foundation. "OWASP Top Ten." https://owasp.org/www-project-top-ten/, 2021.

[5] MITRE Corporation. "MITRE ATT&CK Framework." https://attack.mitre.org/, 2024.

[6] Docker Inc. "Docker: Empowering App Development for Developers." https://www.docker.com/, 2024.

[7] Microsoft. "Playwright: Reliable End-to-End Testing for Modern Web Apps." https://playwright.dev/, 2024.

[8] Colvin, S. "Pydantic: Data Validation Using Python Type Annotations." https://docs.pydantic.dev/, 2024.

[9] Hagberg, A., Schult, D., and Swart, P. "NetworkX: Network Analysis in Python." https://networkx.org/, 2024.

[10] Deng, G., et al. "PentestGPT: An LLM-empowered Automatic Penetration Testing Tool." *arXiv preprint arXiv:2308.06782*, 2023.